GUIA TÉCNICO

Fundamentos de PLN e Linguística Computacional

Propósito — Conteúdo didático autoral — notas técnicas e exemplos independentes. Material de apoio histórico não é reproduzido neste PDF.

Como utilizar — Leia as explicações, execute os exemplos no ambiente indicado e consulte os notebooks correspondentes.

ROTEIRO DO DOCUMENTO

1. Fundamentos e objetivo
2. Modelo e escolhas metodológicas
3. Receitas Python reproduzíveis
4. Interpretação e limitações
5. Exercícios de aplicação
6. Checklist de qualidade

01 | O problema e seu contexto

Fundamentos de PLN e Linguística Computacional é estudado como técnica computacional, com hipóteses explícitas, dados de demonstração e análise crítica dos resultados.

Cuidados metodológicos

Um mesmo termo pode ter significados diferentes; a tokenização com split é apenas aproximação. Estabeleça tarefa, idioma, corpus e critérios de avaliação.

02 | Fundamentos e modelo

Objetivos de aprendizagem

- Distinguir tokens, documentos, corpora e tarefas de PLN.
- Explicar ambiguidade, contexto, domínio e variação do português.
- Identificar cenários supervisionados, não supervisionados e generativos.

Fundamentos conceituais

Uma linguagem natural possui níveis de análise: morfológico (formas de palavras), sintático (relações estruturais), semântico (significados) e pragmático (efeitos de contexto). Um mesmo texto pode conter ambiguidades que um conjunto de regras fixas não resolve. PLN não equivale a apenas remover pontuação: o pré-processamento deve respeitar a tarefa e o significado relevante.

Modelo matemático e interpretação

Um corpus é representado como $D = \{d_1, \ldots, d_n\}$. O pipeline de aprendizagem transforma cada documento d_i em características x_i e, quando há rótulo y_i, ajusta uma função $f(x_i)$ para aproximar y_i. Aprendizagem generativa estima distribuições sobre sequências de tokens, não somente rótulos.

Estudo guiado

Pergunta de investigação: como demonstrar o conceito central com um exemplo pequeno, auditável e reproduzível?

Nas células seguintes, inspecione as entradas, execute os procedimentos e explique com suas próprias palavras o resultado. Ao trocar o texto ou o parâmetro, observe se o comportamento permanece coerente. Esses resultados são ilustrações pedagógicas, não benchmarks.

Implementação reproduzível

O código abaixo usa textos sintéticos ou bibliotecas locais. Não executa downloads nem acessa APIs externas. Exemplos opcionais só rodam quando a dependência já está disponível; a ausência deles não interrompe o notebook.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Interpretação guiada: verifique se o resultado corresponde à fórmula e à hipótese discutidas. Anote o que muda com a entrada e quais conclusões não podem ser generalizadas a partir deste exemplo.

Exercícios orientados

1. Compreensão: explique qual informação o método preserva e qual descarta. Relacione a resposta com os objetivos do capítulo.
2. Experimento: altere uma frase e um parâmetro do código. Registre a diferença e uma hipótese explicativa.
3. Avaliação crítica: descreva uma situação em que a técnica geraria uma resposta inadequada. Proponha uma mitigação.
4. Transferência: conecte o conceito a uma disciplina anterior e identifique o que não precisa ser revisto.

Resolução comentada (diretriz de resposta): o método representa somente um recorte do fenômeno linguístico; mudanças de entrada alteram a saída segundo a regra estudada. Uma boa análise identifica perda de informação, contexto e risco de generalização. Para a resposta específica, justifique com os valores efetivamente obtidos na execução acima.

Erros comuns, limitações e boas práticas

Não confundir fluência com precisão factual. Reconhecer variação regional, ironia, negação e linguagem especializada. Evitar usar os exemplos sintéticos como evidência de desempenho em produção.

- Separe demonstração de produção: datasets sintéticos não demonstram generalização.
- Registre versões das dependências e parâmetros; não inclua credenciais no notebook.
- Consulte a documentação oficial antes de adaptar códigos históricos de 2021.

03 | Cookbook Python

Contar tokens simples

```
textos = ['Banco de dados', 'Banco do parque']
for texto in textos:
    print(len(texto.split()), texto)
```

Contar documentos

```
from collections import Counter
corpus = ['sol e chuva', 'sol e vento']
print(Counter(' '.join(corpus).split()))
```

04 | Interpretação e limites

Um mesmo termo pode ter significados diferentes; a tokenização com split é apenas aproximação. Estabeleça tarefa, idioma, corpus e critérios de avaliação.

05 | Exercícios com respostas orientadoras

1. Altere uma entrada ou parâmetro e descreva o comportamento observado.

Resposta orientadora: mantenha os demais elementos constantes, documente a mudança e explique qual resultado deveria variar pela hipótese escolhida.

2. Cite uma situação em que a técnica falha ou transmite confiança excessiva.

Resposta orientadora: identifique domínio, idioma, quantidade de dados, formato da entrada, possíveis erros sistemáticos e um teste que exponha o limite.

3. Escreva um teste de sanidade que possa ser executado antes de analisar os resultados.

Resposta orientadora: estabeleça forma, tipo, dimensões ou invariantes esperados para os dados, e uma observação que invalidaria a conclusão.

06 | Referências e consulta

Material construído a partir de exercícios autorais da biblioteca, com o conteúdo histórico utilizado apenas como apoio privado. Documentação técnica: Python (docs.python.org), scikit-learn (scikit-learn.org), NLTK (nltk.org), spaCy (spacy.io).

Portfólio GitHub: https://github.com/eduardojr-pereira/data-science-portfolio

Site pessoal: https://eduardo-pereira.webflow.io/